CS 440: Introduction to Artificial Intelligence Fall 2026

Assignment 1: Fast Trajectory Replanning

Due Saturday, October 10, 2026, at 10:00 PM EST

Implement and compare search algorithms for an agent navigating an unknown grid. Prove their guarantees and explain the measured trade-offs.

Work in a team of up to four, or individually. The assignment is worth 100 points: 55 autograded and 45 for written answers and code review. The tasks are on pages 2–4; experiments, submission, and course rules follow on pages 4–5.

Start with four files

Choose Python, C++17, or Java 17+. All use Python 3.10+ to run the supplied tools; C++ and Java students need not write Python. No third-party libraries. The starter supplies sensing, world generation, measurements, plots, and PDF creation. You do not need to read their implementations. Use the API summary in `README.md` and the comments for the part you are working on.

File (Python / C++ / Java)	Implement	Parts
`heap.py` / `Heap`	Binary min-heap	2
`search.py` / `Search`	A*, tie-breaking, learned h-values, weighting	2, 3, 5, 7
`agents.py` / `Agents`	Forward, backward, and adaptive replanning	2, 4, 5
`heuristics.py` / `Heuristics`	Landmark and custom heuristics	6

C++ files use `.hpp`; Java uses `.java`. Paths and APIs are in the README. Also complete `report.txt`. Leave the other files unchanged; Gradescope uses staff copies of the support code. Start by running:

```
python3 demo.py --tour
python3 autograder.py
```

The tour works before you implement anything; open its images in `figures/`. Use `--part 2`, then later part numbers, to check your work incrementally. See `README.md` for small examples and debugging commands.

The world and the search

A state is a cell (x, y). The agent moves north, south, east, or west to an unblocked neighbor; each move costs one. The map and target are fixed. Start and target cells are unblocked. The agent initially knows their locations and learns the blockage status of its four neighbors by sensing.

Under the **freespace assumption**, unobserved cells are treated as free. Sense before the first plan. Plan over the current belief, follow the path while the next cell is known to be free, and replan when that cell is blocked. The supplied movement method records each move and senses automatically. Stop on reaching the target or finding no path in the believed map. Do not access the private true map.

For A*, $g(s)$ is the best discovered cost from the search start, $h(s)$ estimates the remaining cost, and $f(s) = g(s) + h(s)$. Maintain OPEN as your heap and CLOSED as an explicit set. Select minimum-f states, test for the goal before expanding, and relax successor costs. Return a path and its g-values; use the unreachable representation in your starter's README. Use maps for discovered states; do not initialize the whole grid for each search.

A heuristic is **admissible** if $0 \le h(s) \le h^*(s)$, where $h^*(s)$ is the true remaining cost on the map being searched. It is **consistent** if $h(\text{goal}) = 0$ and $h(s) \le c(s, s') + h(s')$ on every edge. With a consistent heuristic, unweighted A* need not reopen CLOSED states. Use Manhattan distance $h(s) = |x_s - x_g| + |y_s - y_g|$ for Parts 2–5.

Tasks: implement, then explain

Answer Q1–Q7 in `report.txt`: at most 520 rendered characters per part, covering all subquestions. Prepare full proofs for review; use macros for measurements.

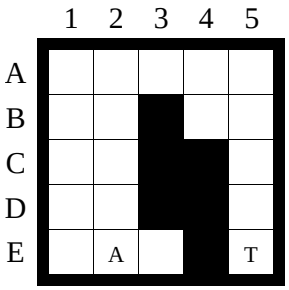
Part 0 — Setup [5 points: autograded]

An untouched starter earns 3/55. Complete the team list and pledge before experiments. Setup credit covers the source checks, unchanged support files, and a complete `report.txt`.

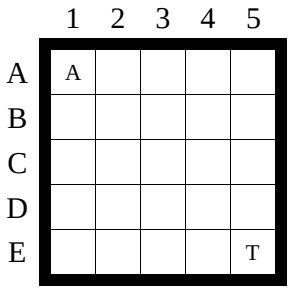
Part 1 — Understanding A* [5 points: written]

a) In Figure 1(a), why does the agent first move east rather than north, given its limited knowledge?

b) Prove that Manhattan distance is consistent for four-directional, unit-cost movement. State the edge inequality and include the goal condition.



(a) Part 1: initial agent A and target T



(b) Part 3: an open grid for tracing ties

Figure 1: Complete maps for the questions. Black cells are blocked; white cells are free.

Part 2 — Repeated Forward A* [15 points: 10 code, 5 written]

Implement the binary heap, then A*, then the forward agent. Search from the agent's current cell to the target. Use Manhattan distance and prefer larger g among equal f values.

Use your own heap; library priority queues and heap algorithms are prohibited. Insertion and minimum extraction must be logarithmic. Justify decrease-key or lazy deletion, including memory cost. Use an explicit CLOSED set; never re-expand within a search.

Written: Prove that the agent either reaches the target or determines unreachability in finite time. Then prove that it makes at most n^2 moves, where n is the number of unblocked cells.

Part 3 — Tie-breaking [15 points: 8 code, 7 written]

Implement both `large_g` and `small_g` tie-breaking. Use the same remaining tie rule, such as insertion order. Trace the rules on Figure 1(b), then compare them on both terrain families.

Written: Why can expansion counts differ when both rules select nondecreasing f and find shortest paths? Relate the measured difference between terrain families to how many cells share an f value.

Tasks: search direction and heuristic reuse

Part 4 — Backward search [15 points: 8 code, 7 written]

Implement the backward agent: search target-to-agent, with h measuring distance to the agent, then reverse the path. Compare both directions on both families, with larger-g ties and identical remaining tie rules.

Written: Use your measurements to explain how new blockages affect each direction. Where does each search start, how does that change between replans, and which regions may be searched again?

Part 5 — Adaptive A* [15 points: 8 code, 7 written]

Implement Adaptive A*. After a successful unweighted search of cost $C = g(\text{goal})$, learn $C - g(s)$ for each expanded s, keeping its largest learned value. Later searches use

$$h_{\text{used}}(s) = \max\{h_{\text{Manhattan}}(s), h_{\text{learned}}(s)\}.$$

Do not learn from a failed search. Compare with Repeated Forward A* on both families.

a) Prove that the updates preserve admissibility and consistency from an initially consistent heuristic. Prove that consistency survives increases in action costs between searches.

b) Explain the measured improvement on each family in terms of the gap between Manhattan distance and the true remaining distance.

c) Interpret the provided paired Wilcoxon signed-rank test for both families: state the null, justify pairing, distinguish significance from effect size, and explain failure to reject. Compare its assumptions with a paired t-test. Consider the *differences*; signed-rank location inference assumes symmetry and independent worlds.

Part 6 — Heuristic design [15 points: 8 code, 7 written]

Implement the differential and custom heuristics. Use the provided BFS helper to build distance tables on the believed map; cache them in the search problem. For a set of landmarks L, define

$$h_L(s) = \max_{l \in L} |d(l, s) - d(l, \text{goal})| .$$

Skip terms with missing distances; use zero if none remain. Distances are from the same table for both endpoints. Tables must support the current search goal.

a) Choose and justify the landmarks and their number. Describe what changes when you vary that choice.

b) Prove admissibility and consistency of h_L. Identify where the triangle inequality and symmetry of distance are used, and why symmetry holds here.

c) A table built at belief version v can be reused at later versions. Prove this preserves admissibility: discovering walls removes edges from the believed graph. Interpret the report's `REBUILD_EVERY` sweep, including precomputation cost. Tables rebuild when their version age exceeds this threshold.

d) Design `h_custom` by combining heuristics. Prove that taking a maximum preserves admissibility and consistency. Why can summing them fail?

Compare known-map and replanning results. Explain how unknown terrain, search length, and precomputation cost affect whether landmark tables are useful.

Tasks: weighting and experiment design

Part 7 — Weighted A* [15 points: 8 code, 7 written]

Implement $f(s) = g(s) + wh(s)$ for $w \geq 1$ through `weight`; $w = 1$ must reproduce Part 2. Use a consistent base heuristic and do not reopen CLOSED states.

a) Prove the w-times-optimal path-cost bound. Expanded g-values need not be exact: wh can be inconsistent. Hint: use consistency along a shortest path to inductively show $g(s) \leq wg^*(s)$ for selected states; g^* is the true distance from the start.

b) Interpret the known-map cost-versus-expansions sweep and choose a weight for a game. Larger weights need not reduce expansions on every instance.

c) Why might per-search savings fail to reduce total replanning work? Consider routes, observations, and replans. Must longer plans encounter obstacles sooner? Separate guarantees from empirical effects.

d) Can Adaptive A* safely learn $C - g(s)$ from weighted searches? Give an argument and explain the implication for combining the algorithms.

Experiments and submission

What the provided experiments measure

Reports use 50 worlds per family, seeded from your team's RUIDs. Local and Gradescope autograders share fixed test seeds.

Family	Terrain and main comparison size
`scatter`	Scattered obstacles, roughly 70% free; 101×101. Manhattan distance is often close to the true distance.
`maze`	Narrow corridors carved by depth-first search, with 20% of dead ends reopened to add loops; roughly 50% free; 51×51. Walls can force long detours.

Distinguish a **single search on a fully known map** from the **replanning loop over a changing belief**. The report compares agents and tie rules on both families, sweeps maze sizes, tests heuristics and weights on known maps, and measures landmark rebuilding in the maze loop. These experiments run automatically; Part 7 additionally asks you to reason about weighting inside the loop.

The framework measures expansions, re-expansions, heuristic probes, runtime, and movement. Different plans can reveal different obstacles. Explain your results, which need not match reference numbers. Visualization commands are in `README.md`.

Report, submission, and code review

1. Complete `report.txt`: members and RUIDs, the pledge, and answers Q1–Q7. Keep the template's keys. Each answer is capped at **520 characters after macro substitution**, with no minimum.
2. Cite measured results with a macro, for example `{{scatter.tie.ratio}}`. List available names and draft values with:
   `python3 autograder.py --macros`. Single digits in prose are allowed; typed results such as `15`, `0.42`, `3x`, and `60%` are rejected.
3. Generate a draft with `python3 make_report.py --quick`. For submission, omit `--quick` to run the full experiment set. Inspect the one-page `report.pdf`; resolve warnings and truncated answers. Regenerate it after changing code, answers, or team members.
4. Run `python3 submit.py`. Upload `submission.zip` to Gradescope through Canvas. It includes your chosen language's code and support files, `language.txt`, `report.txt`, `report.pdf`, and the hidden `.allog`. Add every team member to the Gradescope submission. Late submissions receive zero points.

Code review and independent work

The TAs will schedule a 15-minute team review after the deadline. Every member must explain the submitted algorithms and make a small code change. A part the team cannot explain receives zero, including its autograded points.

Do not share code, notes, or worksheets with other teams. Acknowledge external sources and follow university and department integrity rules; violations may result in course failure. Generative AI may answer general conceptual questions but may not write or revise submitted code or answers. Confirm the pledge.

Keep the run log, `.allog`. Staff may compare code and rerun submissions to check PDF fingerprints. A short log or a mismatch alone does not establish misconduct; discrepancies require investigation.

Optional: D* Lite [+10 points]

Implement D* Lite and compare its runtime and expansions with Adaptive A* on both families, using larger-g ties for Adaptive A*. Reference: Koenig and Likhachev, *Fast Replanning for Navigation in Unknown Terrain*, IEEE Transactions on Robotics 21(3), 354–363, 2005. Contact the TAs to arrange submission and code review; the standard autograder does not grade this extension.